

Rozproszone operacje genomyczne na bazie polars-bio

Sprawozdanie ze stanu prac implementacyjnych

Miłosz Kowalewski

sierpień 2026

Spis treści

1. Cel i zakres	1
2. Stan implementacji	2
2.1 Mechanizm integracji z Ballistą	2
2.2 Mechanizm integracji z Sailem	2
3. Sposób weryfikacji	2
4. Dwa wzorce rozpraszania	3
5. Co wymagało obejść	3
5.1 Łatki widoczności w bibliotece pomocniczej	3
5.2 Węzeł-nośnik dla operacji coverage	4
6. Wyniki negatywne i granice rozwiązania	4
7. Znaleźisko dotyczące polars-bio	4
8. Czego jeszcze nie ma	5
9. Proponowane dalsze kroki	5

1. Cel i zakres

Celem prac było sprawdzenie, czy operacje interwałowe z biblioteki **polars-bio** (overlap, merge, nearest, coverage, subtract) da się wykonywać w sposób rozproszony na dwóch silnikach zapytań — **Apache Ballista** i **LakeSail/Sail** — przy dwóch twardych założeniach:

1. integracja wyłącznie jako **rozszerzenie w czasie działania**, bez forkowania któregośkolwiek z silników;
2. funkcja użytkownika ma **faktycznie wywoływać silnik polars-bio**, a nie reimplementować algorytmu od zera.

Dodatkowym założeniem było wykazanie, że możliwa jest **zmiana planu zapytania** (repartycjonowanie danych według chromosomu) po stronie obu silników.

Niniejsze sprawozdanie opisuje stan po zakończeniu etapu weryfikacji poprawności. Etap pomiarów wydajnościowych nie został jeszcze rozpoczęty — patrz sekcja 8.

2. Stan implementacji

Wszystkie pięć operacji działa na obu silnikach. W Ballistcie wszystkie pięć wykonuje się w sposób rozproszony (plan zapytania jest serializowany i przesyłany do executora); w Sailu wszystkie pięć działa przez natywny UDTF, z zachowaniem równoległości.

Operacja	Ballista (lokalnie)	Ballista (rozproszone)	Sail (UDTF)
overlap	tak	tak — z algorytmem COITrees	tak
merge	tak	tak — hash-shuffle po chromosomie	tak
subtract	tak	tak — dwustronny hash-shuffle	tak
nearest	tak	tak — wzorzec broadcast	tak
coverage	tak	tak — wzorzec broadcast	tak

Poza zakresem pozostają dwie operacje z biblioteki `datafusion-bio-function-ranges` niewchodzące w pierwotną piątkę: `complement` (rozproszenie wykonalne, technicznie proste) oraz `cluster` (rozproszenie **niewykonalne** — uzasadnienie w sekcji 6).

2.1 Mechanizm integracji z Ballistą

Ballista i polars-bio należą do tej samej rodziny technologicznej (obie zbudowane na DataFusion), więc integracja jest bezpośrednia: budowany jest plan zapytania zawierający operator z `datafusion-bio-function-ranges`, a następnie wysyłany do klastra.

Wąskim gardłem okazała się **serializacja planu**. Ballista musi przesłać plan od klienta do koordynatora, a potem do executorów; dla węzłów niestandardowych wymaga to dostarczenia własnych koderów — `LogicalExtensionCodec` dla planu logicznego i `PhysicalExtensionCodec` dla fizycznego. Obydwa zostały zaimplementowane. Istotne jest, że **nie wymagało to modyfikacji Ballisty**: kodery rejestruje się na obiekcie konfiguracji sesji, a silnik sam je odczytuje przy starcie koordynatora i executora.

2.2 Mechanizm integracji z Sailem

Sail należy do innej rodziny — komunikuje się protokołem Spark Connect. Nie udostępnia obecnie mechanizmu wstrzykiwania własnych węzłów planu (projektowany interfejs `SailExtension/FFI` nie jest jeszcze zaimplementowany), więc integracja przebiega przez **natywny UDTF**: Sail grupuje dane po chromosomie, a zarejestrowana funkcja wywołuje z powrotem niezmienione `pb.*()` na każdej grupie.

Warto odnotować, że jest to rozwiązanie **docelowe, a nie tymczasowe** — mieści się w założeniu „bez forkowania” i wykorzystuje oficjalny mechanizm rozszerzeń Saila.

3. Sposób weryfikacji

Przyjęto trzy niezależne warstwy dowodowe, ponieważ sama poprawność wyniku nie dowodzi, że obliczenie faktycznie zostało rozproszone.

Warstwa 1 — poprawność względem wyroczni. Punktem odniesienia jest lokalne `pb.*()` uruchomione na tych samych danych. Porównywane są znormalizowane zbiory krotek, nie surowa kolejność wierszy. Pakiet liczy **42 testy** i przechodzi w całości.

Warstwa 2 — struktura planu rozproszonego. Ballista udostępnia `EXPLAIN ANALYZE` zwracające podział na etapy wraz z drzewem operatorów i metrykami. Każde uruchomienie zrzuca ten plan do pliku, a osobny pakiet 16 asercji sprawdza go automatycznie: liczbę

etapów, obecność `partitioning=Hash([chrom@0], N)`, to czy operator konsumuje dane z `ShuffleReaderExec` (czyli z sieci, a nie z lokalnego skanu), oraz czy etap źródłowy jest równoległy.

Warstwa 3 — poprawność jako dowód rozproszenia. Dane wejściowe rozbito na dwa pliki na tabelę i podzielono celowo: nakładające się interwały $[100, 200)$ i $[150, 300)$ leżą w **różnych** plikach, więc na starcie trafiają do różnych partycji. Poprawny wynik scalenia $[100, 300)$ może powstać wyłącznie wtedy, gdy repartycjonowanie po chromosomie faktycznie przeniosło wiersze między partycjami. Gdyby rozproszenie przestało działać, test nie zwolniłby — **zwróciłby błędny wynik**, i to w sposób jednoznaczny.

Przykładowy zrzut planu dla operacji `merge`:

```
=====SuccessfulStage[stage_id=1, partitions=2]=====
SortShuffleWriterExec: partitioning=Hash([chrom@0], 4)
  DataSourceExec: file_groups={2 groups: [[a_part1.csv], [a_part2.csv]]}

=====SuccessfulStage[stage_id=2, partitions=4]=====
MergeExec: min_dist=0, strict=true
  ShuffleReaderExec: partitioning: Hash([chrom@0], 4)
```

4. Dwa wzorce rozpraszania

Operacje podzieliły się na dwie grupy, co wynika z ich semantyki, nie z decyzji projektowej.

Repartycjonowanie po chromosomie (`merge`, `subtract`). Operacje przetwarzają interwały niezależnie w obrębie każdego chromosomu, więc wystarczy skierować wszystkie wiersze danego chromosomu do jednego executora. `subtract` jest węzłem binarnym, więc wymaga **ko-partycjonowania obu stron** — jego plan ma cztery etapy i dwa niezależne shuffle.

Broadcast mniejszej tabeli (`nearest`, `coverage`). Tu przeszukiwana musi być cała druga tabela, nie jej fragment, więc jest ona przesyłana w całości w ładunku planu (w formacie Arrow IPC), a zrównoleglone jest przetwarzanie tabeli większej. Indeks interwałowego nie da się zserializować (`COITree` nie ma serializacji), ale jest w pełni **odtwarzalny** — executor buduje go lokalnie z przesłanych danych, tą samą funkcją, której użyłby wariant jednomaszynowy.

Ograniczenie tego wzorca: ładunek planu przechodzi przez gRPC z limitem **16 MB**, co stanowi twardy sufit rozmiaru broadcastowanej tabeli. Dla plików adnotacji jest to bez znaczenia, dla dużych zbiorów wymagałoby innego podejścia.

5. Co wymagało obejść

5.1 Łatki widoczności w bibliotece pomocniczej

Węzły planu fizycznego w `datafusion-bio-function-ranges` (`MergeExec`, `SubtractExec`, `NearestExec`, `CountOverlapsExec`) są zadeklarowane bez modyfikatora `pub` i nie mają żadnych konstruktorów ani metod dostępowych — powstają wyłącznie jako literały struktury wewnątrz metody `scan()`. Z zewnątrz biblioteki nie da się ich zatem ani nazwać, ani odczytać, ani skonstruować, co uniemożliwia napisanie kodera planu fizycznego.

Zastosowano lokalną kopię biblioteki z **dwoma łatkami czysto widocznościowymi**: udostępnienie modułu z typem `ColIntervals` oraz dodanie `pub` przy wspomnianych strukturach i ich polach. Łącznie **38 słów pub i 5 re-eksportów, zero linii logiki**. Obie łatki nadają się do zgłoszenia jako pojedynczy, niewielki wkład do biblioteki źródłowej; po ich przyjęciu lokalna kopia przestałaby być potrzebna.

Podkreślenia wymaga, że **nie dotyczy to Ballisty ani Saila** — oba silniki pozostają niezmodyfikowane, zgodnie z założeniem.

5.2 Węzeł-nośnik dla operacji coverage

coverage okazał się jedynym przypadkiem, w którym łatka widoczności nie wystarczyła. Metoda `scan()` materializuje lewą tabelę, przekazuje ją do konstruktora indeksu i **porzuca** — powstały węzeł nie przechowuje ani danych wejściowych, ani nazw ich kolumn, ani flagi trybu. Rozwiązaniem jest przezroczysty dekorator, który deleguje całe zachowanie do węzła biblioteki, ale dodatkowo przenosi to, czego tamten nie pamięta.

6. Wyniki negatywne i granice rozwiązania

Operacja cluster nie poddaje się rozproszeniu. Jej implementacja numeruje znalezione skupiska interwałów globalnie, a synchronizacja odbywa się przez barierę typu rendez-vous działającą w obrębie jednego procesu: każda partycja rejestruje swój wynik cząstkowy i czeka, aż zgłoszą się wszystkie pozostałe. Po rozproszeniu każdy executor otrzymałby własną, odrębną kopię koordynatora, więc plan albo zawisłby w oczekiwaniu na partycje, które nigdy się nie zarejestrują, albo cicho przydzielił kolidujące identyfikatory.

Jest to bloker **semantyczny**, którego żadna zmiana widoczności API nie usuwa. Wniosek wart odnotowania: granica rozpraszalności przebiega nie tam, gdzie kończy się dostępność interfejsu, lecz tam, gdzie algorytm zakłada wspólną pamięć.

Operacja overlap przechodzi przez granicę serializacji, ale bez równoległości hash-partycjonowanej. Kodery działają i algorytm COITrees wykonuje się po stronie executora, jednak węzeł złączenia interwałowego powstaje w trybie nierozstrzygniętym (`PartitionMode::Auto`), a w tym trybie deklaruje brak wymagań co do rozkładu danych — więc repartycjonowanie nigdy nie jest żądane. Przyczyną jest usunięcie z sesji standardowej reguły optymalizatora, która normalnie ten tryb rozstrzyga. Ograniczenie jest udokumentowane asercją, która zasygnalizuje jego zniknięcie.

Sail wymaga Kubernetesa do rzeczywistej wieloprocusowości. Biblioteka ma tylko dwie implementacje zarządcy workerów: jedną uruchamiającą workery jako aktorów w obrębie jednego procesu (używaną **zarówno** przez tryb lokalny, **jak i** przez tryb nazwany `local-cluster`) oraz drugą, tworzącą pody Kubernetes. Potwierdzono to empirycznie — rzut procesów systemu w trakcie działania trybu `local-cluster` wykazał, że wszystkie cztery role (sterownik, dwa workery, serwer Spark Connect) należą do tego samego identyfikatora procesu. Próba uruchomienia lokalnego Kubernetesa nie powiodła się z powodu ograniczeń pamięciowych maszyny testowej (3,5 GB RAM).

7. Znalezisko dotyczące polars-bio

Przez wcześniejszą część prac obserwowano objaw polegający na tym, że przy wykonaniu funkcji użytkownika na więcej niż jednej partycji część wyników cicho zniknęła. Objaw przypisywano początkowo błędowi Saila i obchodzono wymuszeniem pojedynczej partycji, co działało, ale eliminowało całą równoległość.

Seria eksperymentów wykazała, że **atrybucja była nieprawidłowa**. Wszystkie warianty uruchamiano bez wymuszania jednej partycji, po kilka powtórzeń (objaw jest niedeterministyczny):

Wariant	Poprawnych
funkcja czysto pythonowa, bez wywołania polars-bio	3/3

Wariant	Poprawnych
wywołanie <code>pb.merge()</code>	1/3
wywołanie <code>pb.merge()</code> po jawnym wyczyszczeniu kontekstu	0/3
<code>applyInPandas</code> z <code>pb.merge()</code> — odrębna ścieżka kodowa Saila	0/3
wywołanie <code>pb.merge()</code> przez blokadę współbieżności	5/5

Wiersz pierwszy rozstrzyga sprawę: **identyczny kształt zapytania** z funkcją niekorzystającą z polars-bio jest w pełni poprawny, a błąd pojawia się także na zupełnie innej ścieżce kodowej Saila (`applyInPandas`), co wyklucza wyjaśnienie specyficzne dla użytej składni.

Przyczyną jest **globalny, mutowalny kontekst DataFusion w polars-bio**: przy współbieżnym wykonaniu wielu partycji w jednym procesie kolejne wywołania `pb.*()` nadpisują sobie zarejestrowane tabele robocze. Rozwiązanie zastosowane po stronie wywołującej — blokada serializująca dostęp, umieszczona w osobnym module importowalnym — przywraca pełną poprawność przy zachowanej równoległości.

Ograniczenie jest zatem **usuwalne bez modyfikowania polars-bio**, ale warto rozważyć usunięcie go u źródła, ponieważ dotyczy każdego scenariusza wielowątkowego użycia biblioteki, nie tylko opisywanego tutaj.

8. Czego jeszcze nie ma

Etap weryfikacji poprawności można uznać za domknięty. Otwarte pozostają natomiast obszary istotne dla naukowej części pracy:

- **Benchmarki.** Nie wykonano dotąd żadnych pomiarów wydajnościowych. Wszystkie dotychczasowe testy operują na zbiorze pięciu interwałów, dobranym pod kątem wykrywalności błędów, nie skali.
- **Dane rzeczywiste.** Nie użyto jeszcze realnych plików BED/VCF; nierówne rozmiary chromosomów mogą ujawnić problemy niewidoczne na danych syntetycznych.
- **Klaster wielopprocesowy.** Dotychczasowe wyniki dla Ballisty pochodzą z trybu, w którym koordynator i executor działają w jednym procesie. Plan jest tam realnie serializowany i przechodzi przez shuffle, ale nie przez rzeczywistą sieć. Konfiguracja wielokontenerowa jest przygotowana, lecz nieuruchomiona.
- **Integracja wewnątrz polars-bio.** Logika wyboru silnika żyje obecnie **obok** biblioteki, jako osobny moduł. Docelowe wprowadzenie reguły optymalizatora do samej biblioteki nie zostało rozpoczęte — jest to zmiana wymagająca osobnej decyzji.
- **Interfejs pythonowy dla Ballisty.** Strona Ballisty to obecnie program w Rust uruchamiany jako proces, nie funkcja wołalna z Pythona.

9. Proponowane dalsze kroki

Krok 1 — domknięcie etapu lokalnego (bez kosztów zewnętrznych):

- uruchomienie Ballisty jako osobnych procesów przez konfigurację kontenerową (wymaga niewielkiej zmiany w kodzie: połączenia z istniejącym koordynatorem zamiast trybu wbudowanego) — łapie błędy pakowania i konfiguracji sieci zanim zaczną kosztować;
- przygotowanie i lokalna walidacja narzędzia pomiarowego oraz zbioru danych rzeczywistych;
- interfejs pythonowy, potrzebny niezależnie do integracji z polars-bio.

Krok 2 — pomiary w chmurze. Dopiero po powyższym: skalowanie liczby węzłów i rozmiaru danych, oraz uruchomienie Saila w trybie Kubernetes, który lokalnie okazał się nieosiągalny.

Konfiguracja wdrożeniowa dla obu silników jest przygotowana.

Krok 3 — decyzje wykraczające poza implementację, wymagające ustalenia:

1. Źródło danych do benchmarków i ich założenia metodyczne — czy przedmiotem pomiaru są wyłącznie czasy wykonania, czy również np. skalowalność względem liczby węzłów, zużycie pamięci, wolumen danych przesyłanych przez sieć, albo porównanie algorytmów indeksowania interwałów.
2. Forma i termin wprowadzenia zmian do samego polars-bio.
3. Zakres pracy w odniesieniu do operacji complement i cluster.
4. Dostęp do zasobów obliczeniowych na potrzeby pomiarów.